Recursive Insert Saturation

An Observation-Qualified Local Region

Mettapedia Working Notes

31 October 2026

Abstract

Six Lean modules connect an explicit recursive insert-once agenda and a separately defined delta-round engine to the existing finite Horn closure. Every admitted schedule terminates and produces the least closed fact set. This agreement does not imply equal insertion-attempt counts or ordered answers. We prove an exact branch-emission count law, exhibit separating controls, and construct an observer-qualified selector that minimizes a supplied cost model among two eligible execution plans. The result is local: it does not translate arbitrary MeTTa, replace probabilistic or non-clausal reasoning with Horn logic, or certify native runtime behavior.

1 The boundary precedes the algorithm

Let A be an explicitly finite ground fact universe, R a list of positive Horn rule occurrences, and S a list of seed insertion attempts. A rule is the existing `Logic.LP.PropRule`: a finite premise set and one head. The independent reference is

$$C = \mathrm{saturate}(\mathrm{set}(R), \mathrm{set}(S)).$$

The existing `PropositionalChainer` proves that membership in this set is equivalent to its inductive Horn derivability relation, and that C is the least rule-closed set containing the seeds. Neither operational connection proofs nor the connection checker are dependencies here.

This carrier expresses a qualified region, not the surrounding language. The fact store is a set, premise tests are pure membership, and the only negative test suppresses insertion of the same already-present fact. Arbitrary negative premises, external mutation, bags of facts, hidden effects, and evidence revision require other contracts. A finite program does not by itself imply a finite grounding: one successor schema over natural numbers already separates those conditions.

Eligibility and optimality are different obligations. A representation change must preserve the complete declared consumer. Only then may an execution plan be selected by cost. The results below establish permission inside this finite region; they do not establish that Horn compilation is optimal or appropriate for every source computation.

2 Recursive control remains explicit

`RecursiveInsertAgenda` has two tasks: insert a fact, or check a rule. Its state consists of known facts K, a pending task list, and an event log. An already-present insertion records a present event and schedules nothing. A fresh insertion adds its fact, records a fresh event, and schedules a check for every rule occurrence mentioning that fact. A check tests its premises against the facts known *when it executes*; if enabled, it schedules an insertion of its head. Empty-premise

rules are checked initially. Seeds start as pending insertions, not as known facts whose firing is omitted.

Depth-first scheduling places children before the remaining continuation; FIFO places them after it. The general policy must preserve the exact task multiset. It may neither discard tasks nor manufacture extra occurrences. Rule occurrences and seed attempts are lists, so duplicates remain visible to the operational log even though the reference semantics is set-valued.

For $m = |R|$ and $N = |A|$, insertion tasks have weight 1 and check tasks weight 2. Every transition strictly decreases

$$\Phi(K, Q) = (2m + 1)(N - |K|) + \sum_{t \in Q} \text{weight}(t).$$

A fresh fact can create at most m checks. Its unseen-fact credit pays for those checks; an enabled check replaces weight 2 by an insertion of weight 1. Thus $\Phi + 1$ units of fuel suffice. This is an agenda-transition bound, not a grounding, indexing, instruction-count, or elapsed-time bound.

Fuel exhaustion on a nonempty agenda returns a pause carrying the entire remaining state. The exact resumption law is

$$\text{run}(a + b, s) = \text{resume}\big(b, \text{run}(a, s)\big).$$

An empty agenda is recognized as completed even with zero remaining fuel. For the admitted finite policies, all work finishes; no extra assumption that a pending obligation will eventually be selected is hidden in the closure theorem. An external scheduler that stalls or cancels is not such a completed execution.

`RecursiveInsertIncremental` handles entry over a baseline B that is already closed under the same rules. It begins with B known and schedules only the new seed attempts. The final store is the closure of $B \cup S$; old facts do not need to fire again. The fresh-weight fold is over $C \setminus B$, not over C. The proof also connects this incremental run to delta saturation initialized with $B \cup S$. A control with a initially stored, rule $a \Rightarrow b$, and no pending tasks shows why an unclosed baseline cannot be admitted this way: execution stops without deriving b. Successive source checkpoints remain separate region boundaries.

3 Why completion implies the least closure

`RecursiveInsertSaturation` proves two independent invariants. Soundness states that known facts and pending insertions belong to C, and pending checks are authorized rule occurrences. Coverage states that every seed is known or pending, and an enabled rule has its head known, a pending head insertion, or a pending rule check.

A newly enabled rule must mention the just-inserted fact, so its check is scheduled. A previously enabled rule retains an outstanding obligation or gets its head inserted. This is the key completeness argument: removing a task does not silently remove the work it represented.

At completion, seed coverage gives $S \subseteq K$ and rule coverage makes K closed. Leastness gives $C \subseteq K$; soundness gives $K \subseteq C$. Consequently every completed permitted schedule has exactly $K = C$. This is final-set confluence, not confluence of the event trace.

4 A separately defined semi-naive engine

`SemiNaiveInsertSaturation` uses a known set K and a fresh delta D. It first selects candidate rules whose premise sets meet D, plus the empty-premise rules. Only candidates have their full

premise subsets tested. Let H be the enabled candidate heads. The next state is

$$D' = H \setminus K, \qquad K' = K \cup D'.$$

Initially $K = D = \mathrm{set}(S)$. The preserved invariant says that an enabled noncandidate already has its head in K. Therefore one delta round's known set equals the existing naive consequence step. Induction identifies the known sets at every round, so the existing finite closure bound applies. One further round has empty delta.

The candidate set is a subset of the program. No claim is made that a native index has already been implemented, or that scanning candidates is always cheaper than another plan. Empty-premise rules remain eligible in subsequent rounds; they cannot contribute a fresh head twice. Operational duplicate attempt counts are not reconstructed by this set engine.

5 Exact counts, and an explicit failure of count confluence

`RecursiveInsertObservations` assigns each fact a fixed local list of fresh-branch answers $F(x)$ and present-branch answers $P(x)$. Call emissions concatenate. For an actual completed event log L,

$$|\,\mathrm{answers}(L)| = \sum_{x \in C} |F(x)| + \sum_{\mathrm{present}(x) \in L} |P(x)|.$$

The proof maintains the stronger invariant that the sum of arbitrary natural fresh-event weights equals the same weight fold over the known set. Thus the first term is schedule independent; the second retains attempt multiplicity. When $P(x) = []$ for every x, the answer count is a closure fold. Ordered answer lists can still differ.

These are explicit local emission semantics. They do not assert that every evaluator return convention or every top-level `collapse` has this form. For the inspected LUBM spelling, a source bridge must account for recursive `fire-ty/fire-tr` returns, reverse views, intermediate observers, and the active profile before using an answer-count specialization.

The separating positive program has seeds a, b and one rule $\{a, b\} \Rightarrow c$. DFS checks after a while the rule is disabled, then checks after b and inserts c. FIFO first inserts both seeds; both queued checks then succeed. Both finish with $\{a, b, c\}$, but DFS makes three insertion attempts and FIFO four. Counting every call therefore distinguishes them. Another control with $a \Rightarrow c$ gives fresh orders a, c, b and a, b, c, even though duplicate emissions are silent.

Controls also cover cycles, empty-premise rules, duplicate seeds and rules, cutoff versus completion, negative premises, and removal during propagation. For every nonempty finite natural-number set K,

$$K \cup \mathrm{succ}(K) \neq K.$$

Finite syntax cannot supply the missing finite-universe proof.

6 An observer-qualified local selector

`RecursiveInsertRegion` offers two plans: the actual bounded depth-first agenda and the independent delta engine. Requests may observe final facts, a fresh-weight count, attempt count, an ordered trace, or a pair of requests. The delta plan is eligible only for final facts and fresh-weight counts; paired requests require both components to be eligible. Unsupported delta observations are declined, never treated as false or as an empty answer.

The refinement theorem equates every eligible plan's observation with that of recursive execution. The selector first checks eligibility, then compares supplied natural-number costs. It proves

$$\text{cost}(\text{choose}(d, c)) \leq \text{cost}(p) \quad \text{for every eligible } p.$$

This is genuine least-cost selection *within the declared two-plan model*. Supplying realistic costs, proving source admission, extending the alternatives, and relating model costs to native execution are separate work. Even zero delta cost cannot override an ordered-trace request. Conversely, if recursive execution is cheaper, it is retained for an otherwise eligible final-set query.

7 Qualification and remaining seams

All six mathematical modules and the joined audit are checked through the project's Lean build procedure, with warnings treated as errors on source checks. The owner-module audit inventories generated helpers as well as authored declarations. The only transitive primitive axioms are `propext`, `Classical.choice` and `Quot.sound`; runtime compiler counterparts are listed separately and are not mathematical proof credit.

The native compiler still needs a profile-qualified source extraction theorem or differential admission checks, a complete observer contract, and implementation qualification. Full given-clause saturation with deletion and subsumption is not this insert-only closure. Nor does a set of derived facts determine PLN evidence provenance, revision or event probabilities. Lattice-valued relaxation, value-equality keys, and the broader structured reasoning developments remain separate packages.